



전자전기컴퓨터설계실험II

실험 후 보고서

조합회로 실습

실험 01-10

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 09월 21일
대상	논리 게이트 · 전가산기 · 4비트 가산기 · 4비트 감산기 · 4비트 크 기 비교기 · 4:1 멀티플렉서 · 1:8 디멀티플렉서 · 8:3 인코더 · 3:8 디코더 · 7세그먼트 디코더
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab1>

제출 태그: [lab1-post-v1](#)

커밋 해시: [fa8db5660b730e5cd8f65d37cb32dcefdb58f1d5](#)

1. 실험 목적

본 실험의 목적은 Verilog HDL을 이용하여 10개의 조합논리회로를 설계하고 그 동작을 검증하는 것이다. 각 회로를 Vivado 2026.1에서 시뮬레이션하고, 테스트 벤치로 계산한 예상 값과 실제 출력 파형을 비교한다. 이후 FPGA 보드에서 회로를 시연하여 시뮬레이션 결과와 실제 하드웨어 동작의 일치 여부를 확인하는데 초점을 둔다.

2. 관련 이론

FPGA 와 Vivado 검증 과정

FPGA는 비트스트림을 이용하여 내부 논리 구조를 구성할 수 있는 재프로그래밍 가능한 소자이다. 이번 실험에서는 Verilog 설계 파일과 SystemVerilog 테스트벤치를 이용하여 Behavioral Simulation을 수행하였다.

테스트 벤치 검증 방식

각 테스트벤치는 가능한 입력 조합을 자동으로 인가하고, 설계 모듈의 실제 출력과 테스트벤치 내부에서 계산한 예상 값을 비교한다. 파형의 n은 진행된 입력 번호, checked는 검사가 완료된 입력 수, expected는 현재 입력에 대한 예상 출력이다. 최종적으로 n과 checked가 같고 pass가 출력되면 모든 입력 조합이 정상적으로 검증된 것으로 판단한다.

3. 실험 환경 및 공통 검증 방법

실험 환경

구분	내용
운영체제	Windows 11 64-bit
FPGA 설계 도구	AMD Vivado 2026.1
설계언어	Verilog HDL
테스트벤치	SystemVerilog
FPGA 소자	xc7s75fpga484-1
회로 종류	조합논리회로
결과 관리	Git 및 GitHub

공통 실행 순서

1. Design Source와 Simulation Source 등록
2. Behavioral Simulation 실행
3. PASS 및 파형 확인
4. Synthesis 실행
5. Implementation 실행
6. DRC 및 타이밍도 확인

7. Generate Bitstream 실행

8. bitstream SHA-256 계산

9. 결과를 Github에 기록

실험별 설계파일

실 험	Design Top	Simul Top	XDC
1	Logic_gate	Tb_Logic_gate	Logic_gate.xdc
2	Full_adder	Tb_Full_adder	Full_adder.xdc
3	Adder_4bit	Tb_Adder_4bit	Adder_4bit.xdc
4	Sub_4bit	Tb_Sub_4bit	Sub_4bit.xdc
5	Compare_4	Tb_Compare_4	Compare_4.xdc
6	Mux_4x1	Tb_Mux_4x1	Mux_4x1.xdc
7	Demux_1x8	Tb_Demux_1x8	Demux_1x8.xdc
8	Encoder8x3	Tb_Encoder8x3	Encoder8x3.xdc
9	Decoder3x8	Tb_Decoder3x8	Decoder3x8.xdc
1	Seg_decode	Tb_Seg_decode	Seg_decoder.xd
0	r	r	c

실험별 Vivado 결과 커밋

실 험	커밋 해시
1	78251870b38fbcc60f7c42a1fa7a22a3f6a80f8e
2	5bad7a5e2a85508492a8a2233662a15d3f492ba4
3	8db89731a368d7e47d90aeeca2f0adb54f9e9daa
4	38ce3542b423b3d2d09c1d35c903065d3e572648
5	a9aa8c246b84074e8ab82d9e58220f6f046664d2
6	32a8ae15e39e7fb5354bfdfbbc7f82e745d228c7
7	a40623c724d809f98c0ced0d247a10ad31704582
8	cbbcb9349d58696dc49c03a4cd795e0b3a1debddc
9	713684c687a4dec6317056bec9d79ded524bc92d
10	efee8b4d57f6a699501afe4745f0dfff91bfd0

4. 실험1 결과 · 논리 게이트

목적 및 설계

실험 1에서는 두 입력 a, b에 대해 AND, OR, XOR 연산을 수행하는 조합논리회로를 구현하였다. 출력 x는 AND, y는 OR, z는 XOR 연산 결과를 나타낸다.

Vivado 실험사진

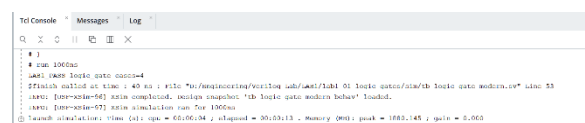


그림 1: 실험 1 Vivado 시뮬레이션 PASS

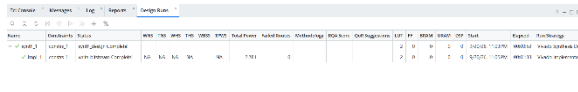


그림 2: 실험 1 논리 게이트 합성 및 구현 완료 결과

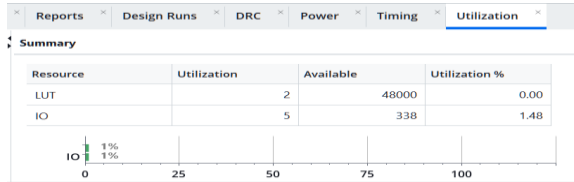


그림 3: 실험 1 논리 게이트 FPGA 자원 사용량

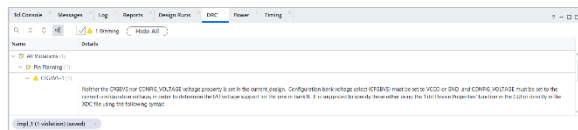


그림 4: 실험 1 논리 게이트 DRC 검사 결과

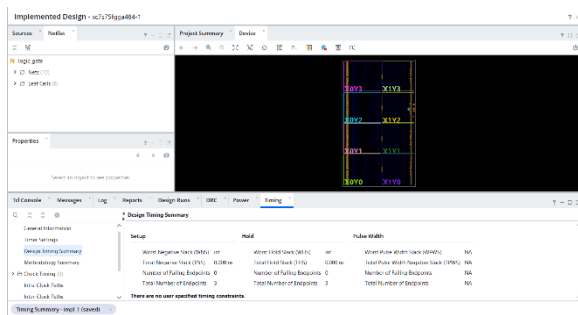


그림 5: 실험 1 논리 게이트 타이밍 분석 결과



그림 6: 실험 1 논리 게이트 Bit Hash 확인

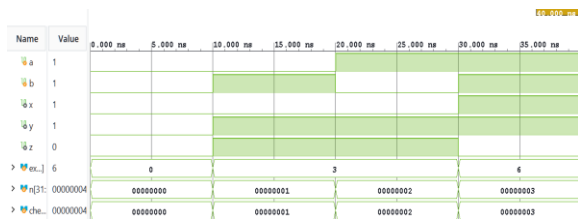


그림 7: 실험 1 입력 조합에 따른 출력 파형

파형 해석 및 결과

입력: a, b가 00-01-10-11 순서로 10ns마다 변화였다. 출력 x, y, z는 각각 AND, OR, XOR 동작을 정확하게 나타냈고, 이는 예상 값과 실제 출력이 모두 일치함을 나타낸다. 또한, 시뮬레이션 및 합성, 비트스트림 생성도 완료되었으며 DRC 오류는 발생하지 않았다.

5. 실험2 결과 · 전가산기

목적 및 설계

실험 2에서는 두 1비트 입력 a, b와 이전 자리올림 입력 Cin을 더하여 합 s와 자리올림 Cout을 출력하는 전가산기를 구현하였다. 두개의 반가산기를 연결하여 전가산기를 구성하였다.

Vivado 실험사진



그림 8: 실험 2 Vivado 시뮬레이션 PASS

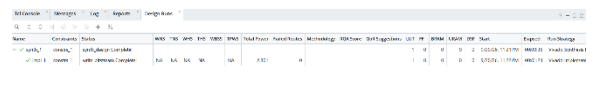


그림 9: 실험 2 전가산기 합성 및 구현 완료 결과

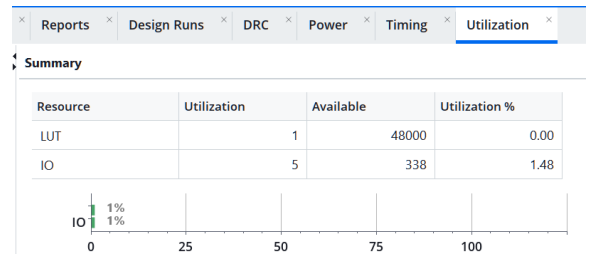


그림 10: 실험 2 전가산기 FPGA 자원 사용량

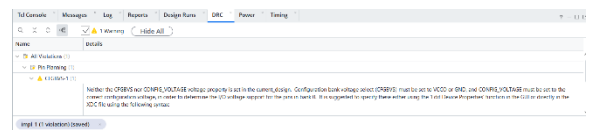


그림 11: 실험 2 전가산기 DRC 검사 결과

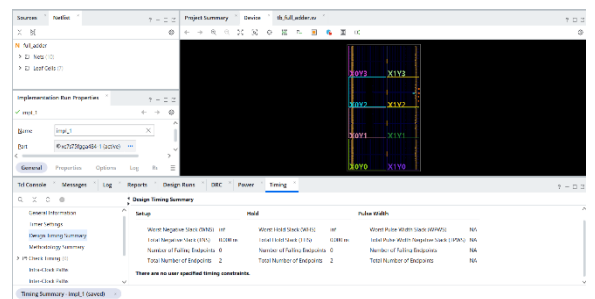


그림 12: 실험 2 전가산기 타이밍 분석 결과



그림 13: 실험 2 전가산기 Bit Hash 확인

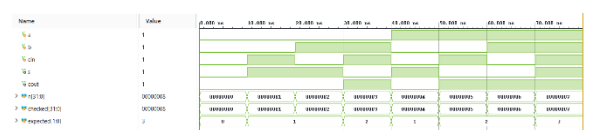


그림 14: 실험 2 입력 조합에 따른 출력 파형

파형 해석 및 결과

a, b, Cin의 8가지 입력 조합이 10ns마다 변화였다. 세 입력의 합이 0 또는 1이면 Cout=0, 합이 2 또는 3이면 Cout=1이 되었다. 또한 출력 s는 전체 합의 최하위 비트를 나타내었고, 이는 예상 값과 일치하였다.

6. 실험3 결과 · 4비트 가산기

목적 및 설계

두 4비트 입력 a와 b를 더하여 하위 4비트 합 s와 최상위 자리올림 Cout을 출력하는 가산기를 구현하였다.

Vivado 실험사진

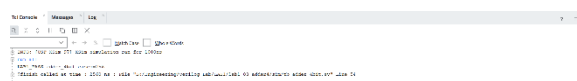


그림 15: 실험 3 Vivado 시뮬레이션 PASS



그림 16: 실험 3 4비트 가산기 합성 및 구현 완료 결과

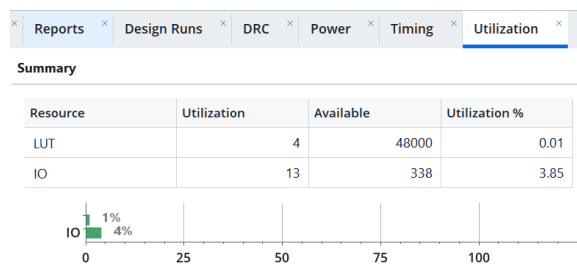


그림 17: 실험 3 4비트 가산기 FPGA 자원 사용량

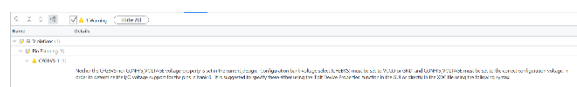


그림 18: 실험 3 4비트 가산기 DRC 검사 결과

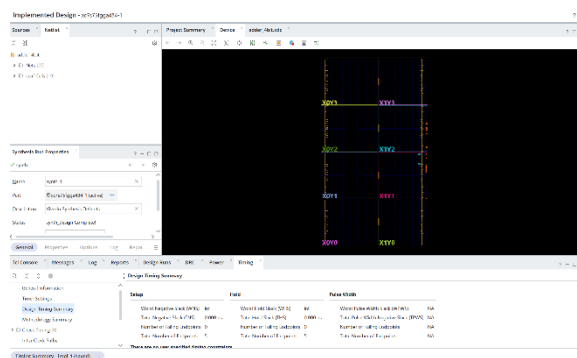


그림 19: 실험 3 4비트 가산기 타이밍 분석 결과



그림 20: 실험 3 4비트 가산기 Bit Hash 확인

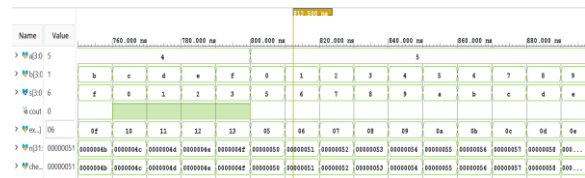


그림 21: 실험 3 입력 조합에 따른 출력 파형

파형 해석 및 결과

두 4비트 입력 a와 b의 256가지 조합을 검사하였다. a=4, b=8일 때 계산 결과는 15이므로 s=F, Cout=0이 된다. 사진에서 보이는 경우인 a=5, b=1에서는 s=6, Cout=0, Expected=06으로 실제 값과 예상 값이 일치하였다.

7. 실험4 결과 · 4비트 감산기

목적 및 설계

두 4비트 입력 a와 b를 빼서 차 d를 출력하고, $a < b$ 인 경우 빌림 출력 bor를 1로 출력하도록 구현하였다.

Vivado 실험사진



그림 22: 실험 4 Vivado 시뮬레이션 PASS

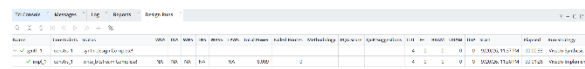


그림 23: 실험 4 4비트 감산기 합성 및 구현 완료 결과

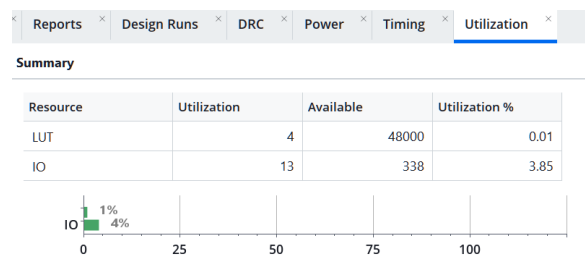


그림 24: 실험 4 4비트 감산기 FPGA 자원 사용량

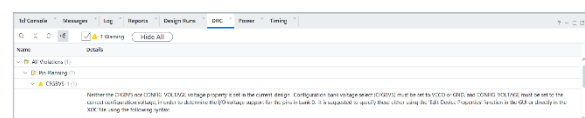


그림 25: 실험 4 4비트 감산기 DRC 검사 결과

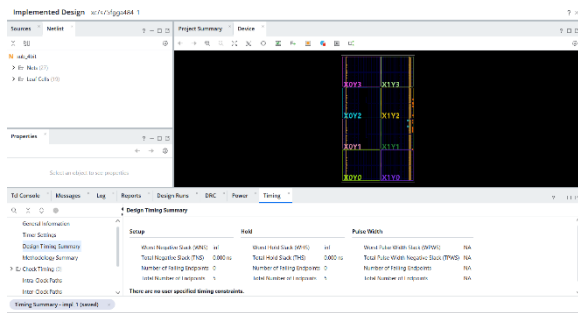


그림 26: 실험 4 4비트 감산기 타이밍 분석 결과

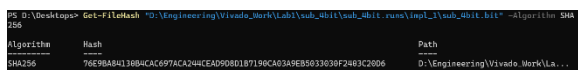
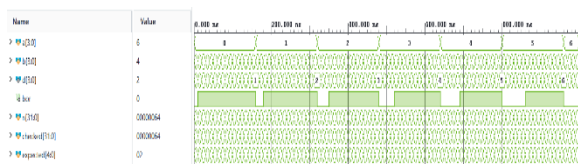


그림 27: 실험 4 4비트 감산기 Bit Hash 확인

그림 28: 실험 4 입력 조합에 따른 출력 파형
파형 해석 및 결과

입력 a는 느리게 증가하고 각 a에 대해 b 값이 빠르게 F가 지 변화하는 파형을 보였다. 사진에서 보이는 경우인 a=6, b=4일 때 d=2, bor=0, expected=02가 출력되어 예상 값과 일치하였다.

8. 실험5 결과 · 4비트 비교기

목적 및 설계

두 4비트 입력 a와 b의 크기를 비교하여 $a > b$, $a = b$, $a < b$ 관계를 3비트 출력으로 나타내는 비교기를 구현하였다.

Vivado 실험사진



그림 29: 실험 5 Vivado 시뮬레이션 PASS

그림 30: 실험 5 4비트 비교기 합성 및 구현 완료 결과

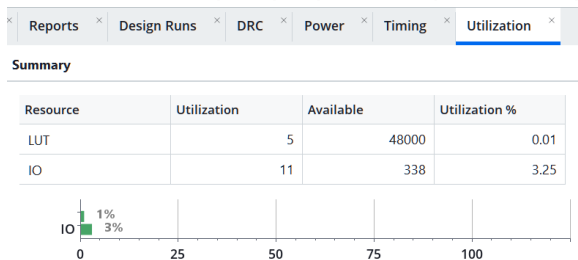


그림 31: 실험 5 4비트 비교기 FPGA 자원 사용량

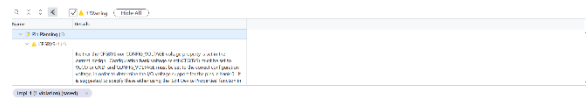


그림 32: 실험 5 4비트 비교기 DRC 검사 결과

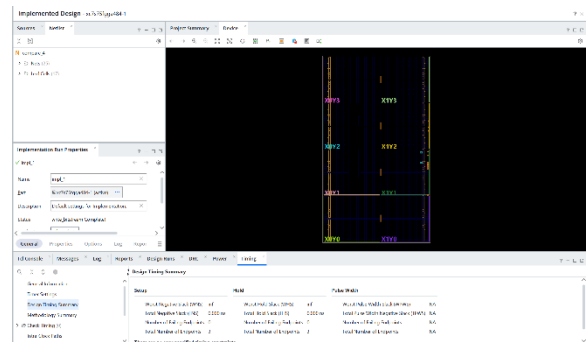


그림 33: 실험 5 4비트 비교기 타이밍 분석 결과

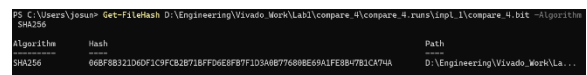


그림 34: 실험 5 4비트 비교기 Bit Hash 확인

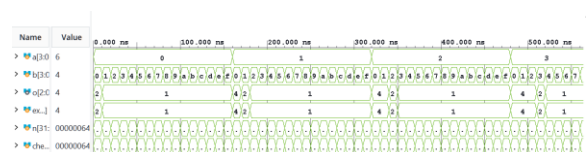
그림 35: 실험 5 입력 조합에 따른 출력 파형
파형 해석 및 결과

사진 파형에서 확인할 수 있듯이 a 값에 대해 b가 증가할 때 a가 큰 구간에서는 출력 100, 같은 구간에서는 010, b가 a보다 큰 구간에서는 001로 변화하였다. 그 예로 a=6, b=4이므로 $a > b$ 이며 출력 o=4, 즉 100으로 나타나 예상 값과 일치하였다.

9. 실험6 결과 · 4:1 멀티플렉서

목적 및 설계

4비트 데이터 입력 i[3:0] 중 선택 신호 s[1:0]이 지정하는 한 비트를 출력 z로 전달하는 4:1 멀티플렉서를 구현하였다.

Vivado 실험사진

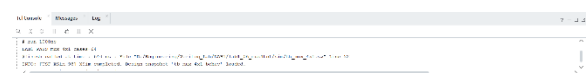


그림 36: 실험 6 Vivado 시뮬레이션 PASS



그림 37: 실험 6 4:1 멀티플렉서 합성 및 구현 완료 결과

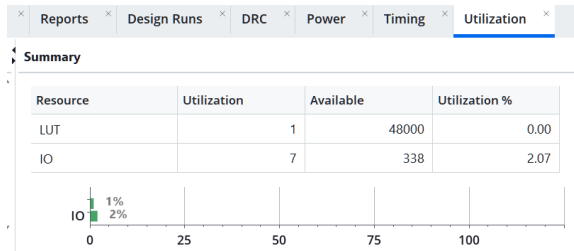


그림 38: 실험 6 4:1 멀티플렉서 FPGA 자원 사용량

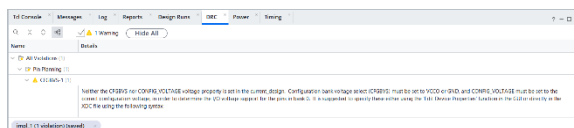


그림 39: 실험 6 4:1 멀티플렉서 DRC 검사 결과

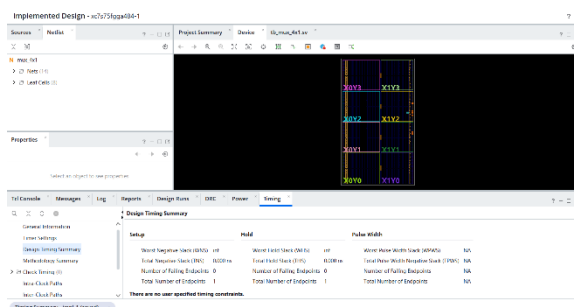


그림 40: 실험 6 4:1 멀티플렉서 타이밍 분석 결과



그림 41: 실험 6 4:1 멀티플렉서 Bit Hash 확인



그림 42: 실험 6 입력 조합에 따른 출력 파형

I=1000인 상태에서 s가 00-01-10-11로 변하면 각각 i의 최상위 비트부터 순서대로 선택되므로 출력 z는 1-0-0-0으로 변한다. Vivado 파형에서 실제 출력과 expected가 모든 구간에서 일치하였다.

10. 실험7 결과 · 1:8 디멀티플렉서

목적 및 설계

하나의 입력 i 를 선택 신호 $s[2:0]$ 에 따라 8개의 출력 중

한 곳으로 전달하는 디멀티플렉서를 구현하였다.

Vivado 실험사진



그림 43: 실험 7 Vivado 시뮬레이션 PASS

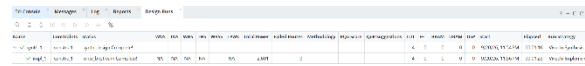


그림 44: 실험 7 1:8 디멀티플렉서 합성 및 구현 완료 결과

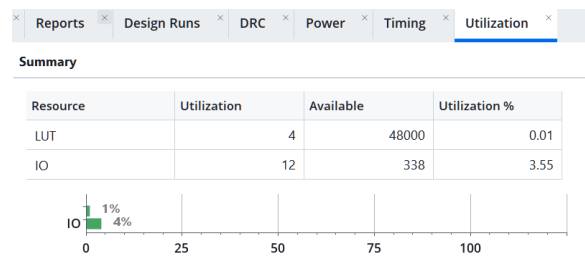


그림 45: 실험 7 1:8 디멀티플렉서 FPGA 자원 사용량

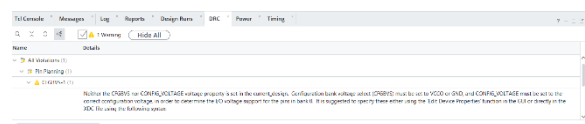


그림 46: 실험 7 1:8 디멀티플렉서 DRC 검사 결과

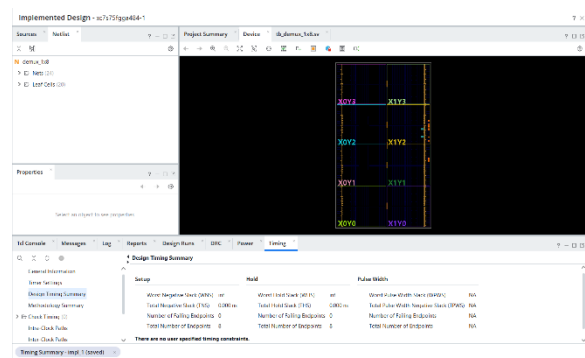


그림 47: 실험 7 1:8 디멀티플렉서 타이밍 분석 결과

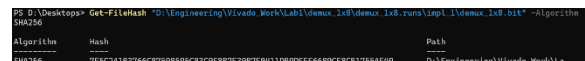


그림 48: 실험 7 1:8 디멀티플렉서 Bit Hash 확인



그림 49: 실험 7 입력 조합에 따른 출력 파형

파형 해석 및 결과

입력 i=0인 처음 80ns에는 선택 신호와 관계없이 출력이 00으로 유지되었다. 이후 i=1인 상태에서는 s가 0에서 7로 증가하는 거에 따라 출력은 80-40-20-10-08-04-02-01로 한 비트씩 이동하는 파형을 보였다. 이는 예상한 디멀티플렉서 동작과 일치하였다.

11. 실험8 결과 · 8:3 인코더

목적 및 설계

8비트 한 입력을 3비트 이진 코드로 변환하는 8:3 인코더를 구현하였다. 유효한 하나의 정해진 입력이 아닌 경우 기본값 000을 출력하도록 설계하였다.

Vivado 실험사진



그림 50: 실험 8 Vivado 시뮬레이션 PASS

Resource	Utilization	Available	Utilization %
LUT	5	48000	0.01
IO	11	338	3.25

그림 51: 실험 8 8:3 인코더 합성 및 구현 완료 결과

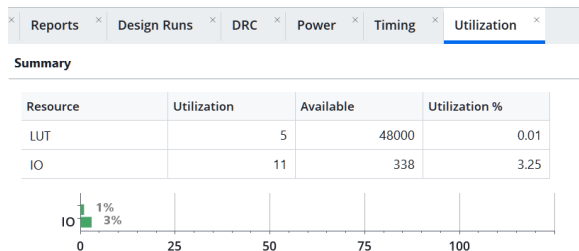


그림 52: 실험 8 8:3 인코더 FPGA 자원 사용량

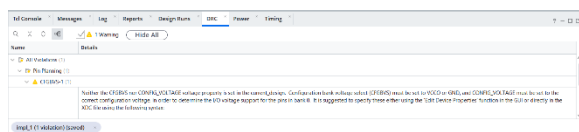


그림 53: 실험 8 8:3 인코더 DRC 검사 결과

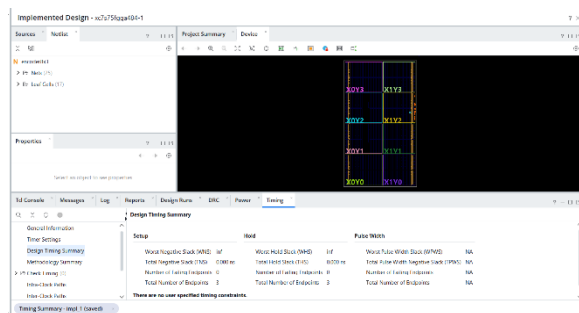


그림 54: 실험 8 8:3 인코더 타이밍 분석 결과

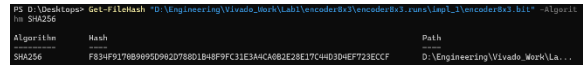


그림 55: 실험 8 8:3 인코더 Bit Hash 확인

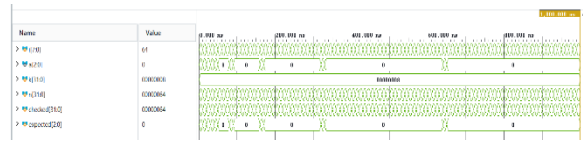


그림 56: 실험 8 입력 조합에 따른 출력 파형

파형 해석 및 결과

유효한 한 입력에 대해 80, 40, 20, 10, 08, 04, 02, 01일 때 각각 0, 1, 2, 3, 4, 5, 6, 7로 인코딩 되었다. 1이 없거나 두 개 이상인 비정상적인 입력에 대해서 기본값인 000이 출력되었다. 이는 모든 유효 입력과 무효 입력에 대해 예상 값과 출력 값이 일치하는 결과를 보였다.

12. 실험9 결과 · 3:8 디코더

목적 및 설계

3비트 입력 a, b, c를 이용하여 8개 출력 중 하나만 1이 되는 3:8 디코더를 구현하였다.

Vivado 실험사진



그림 57: 실험 9 Vivado 시뮬레이션 PASS

Resource	Utilization	Available	Utilization %
LUT	4	48000	0.01
IO	11	338	3.25

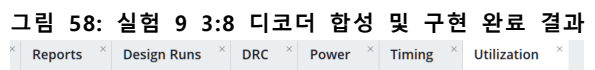


그림 58: 실험 9 3:8 디코더 합성 및 구현 완료 결과

Resource	Utilization	Available	Utilization %
LUT	4	48000	0.01
IO	11	338	3.25

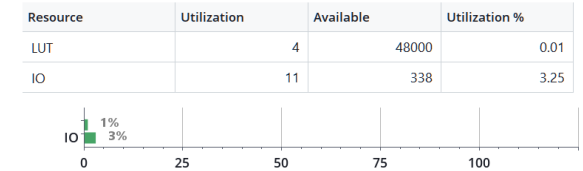


그림 59: 실험 9 3:8 디코더 FPGA 자원 사용량

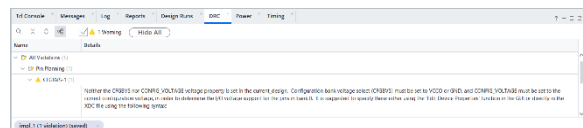


그림 60: 실험 9 3:8 디코더 DRC 검사 결과

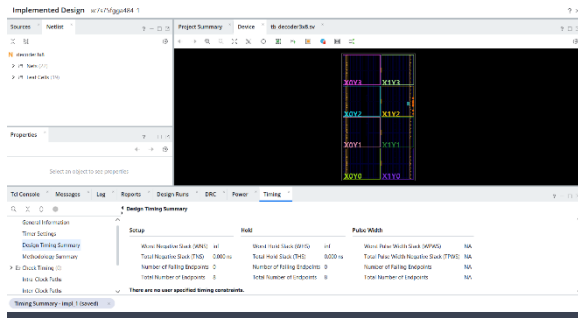


그림 61: 실험 9 3:8 디코더 타이밍 분석 결과

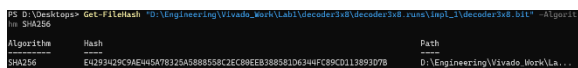
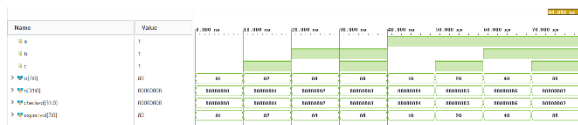


그림 62: 실험 9 3:8 디코더 Bit Hash 확인

그림 63: 실험 9 입력 조합에 따른 출력 파형
파형 해석 및 결과

입력 abc가 000에서 111로 증가할 때 출력은 01-02-04-08-10-20-40-80으로 변화하였다. 이는 00000001을 입력 값 만큼 왼쪽으로 이동한 출력이다. 8가지 입력 조합 모두 실제 출력과 expected가 일치하였다.

13. 실험10 결과 · 7세그먼트 디코더

목적 및 설계

4비트 입력 bcd를 7세그먼트 표시장치에서 0부터 F까지 수로 표시할 수 있는 8비트 세그먼트 데이터로 변환하고자 하였다.

Vivado 실험사진



그림 64: 실험 10 Vivado 시뮬레이션 PASS

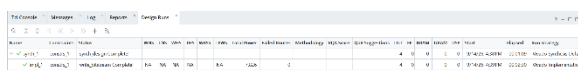


그림 65: 실험 10 7세그먼트 디코더 합성 / 구현 완료

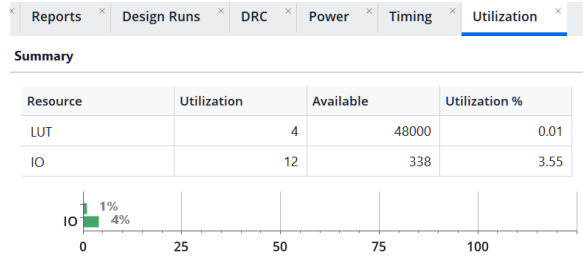


그림 66: 실험 10 7세그먼트 디코더 FPGA 자원 사용량



그림 67: 실험 10 7세그먼트 디코더 DRC 검사 결과

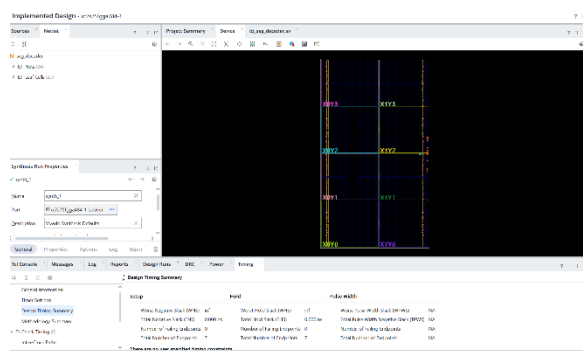
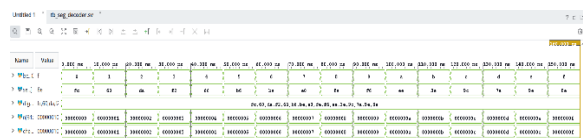


그림 68: 실험 10 7세그먼트 디코더 타이밍 분석 결과



그림 69: 실험 10 7세그먼트 디코더 Bit Hash 확인

그림 70: 실험 10 입력 조합에 따른 출력 파형
파형 해석 및 결과

4비트 입력 bcd 값이 바뀔수록 출력도 따라 변화하였다. 사진에서 보이는 경우를 예로들면 0일 때 FC, 1일 때 60으로 출력된다. 이는 세그먼트 패턴을 통해 비교해보면 파형의 실제 출력과 테스트 벤치의 예상 결과가 모든 입력에서 일치하는 것을 알 수 있다.

14. FPGA 시연 · 실험 5 - 4비트 비교기

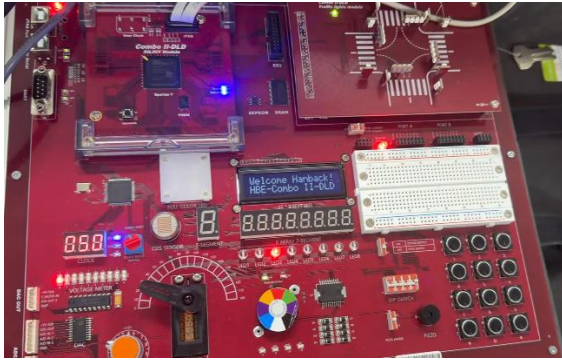


그림 71: 실험 5 4비트 비교기 FPGA 보드 시연

실험 5의 FPGA 보드 시연을 조별로 수행하였다. 보드의 입력 장치를 이용하여 두 4비트 입력 $a[3:0]$ 과 $b[3:0]$ 을 변경하고, 두 수의 대소관계에 따른 출력 $o[2:0]$ 을 LED로 확인하였다. A가 b보다 큰 경우 100이 되어 LED1이 점등되고 같은 경우는 LED2가, b가 더 큰 경우 LED3이 점등되는 것을 확인할 수 있었다. 이는 Vivado 파형에서 확인한 예상 결과와 일치하였다.

시연 영상: <https://github.com/a40093632-ai/ece2-lab1/blob/main/evidence/05/board/videos/demo.mp4>

15. FPGA 시연 · 실험 6 - 4:1 멀티플렉서



그림 72: 실험 6 4:1 멀티플렉서 FPGA 보드 시연

실험 6의 FPGA 보드 시연을 조별로 수행하였다. 데이터 입력 $i[3:0]$ 은 보드의 스위치에 연결하고, 선택 입력 $s[1:0]$ 은 버튼을 이용하여 동작 시켰다. 출력 z 는 LED1을 통해 확인하였다. 선택 입력이 00, 01, 10, 11일 때 각각 입력 값의 최상위 비트부터 출력으로 전달된다. 그 예로 $i=1000$ 이고 $s=00$ 일 때 $i[3]=1$ 이 선택되어 LED1이 점등되었다. 이는 Vivado 파형에서 확인한 변화와 동일했다.

시연영상: <https://github.com/a40093632-ai/ece2-lab1/blob/main/evidence/06/board/videos/demo.mp4>

16. 문제 발생 및 해결 과정

전가산기 하위 모듈 인식 오류

실험 2 전가산기의 Vivado Simulation 실행 과정에서 오류가 발생하였다. 그 원인은 설계파일인 `full_adder.v`는 내부에 두개의 `half_adder` 모듈을 포함하는데 설계파일에 반가산기 관련 파일을 포함하지 않았기 때문에 발생한 것이다. 이를 통해 상위 모듈이 다른 하위 모듈을 사용하는 경우 해당 하위 모듈 파일도 함께 설계 소스에 포함해야 한다는 것을 확인하였다.

Vivado 합성 실패 오류

실험을 하나씩 Vivado에서 실행하는 과정에서 합성이 정상적으로 진행되지 않고 fail이 계속 나타났다. 그 원인은 windows에 설정된 컴퓨터 이름에 한글 문자가 포함되어 있어 관련된 환경 정보를 제대로 처리하지 못해서 발생했던 것이다. 이를 통해 Vivado 프로젝트를 구성할 때 프로젝트 경로 뿐만 아니라 컴퓨터 이름과 사용자 환경에도 한글이나 특수문자가 포함되지 않도록 주의해야 함을 확인하였다.

17. 예상 값 · Vivado · 보드 결과 비교

실험	검증 조합	Vivado 결과	보드 시연	최종
1	4	예상과 일치	미선택	정상
2	8	예상과 일치	미선택	정상
3	256	예상과 일치	미선택	정상
4	256	예상과 일치	미선택	정상
5	256	예상과 일치	선택	정상
6	64	예상과 일치	선택	정상
7	16	예상과 일치	미선택	정상
8	256	예상과 일치	미선택	정상
9	8	예상과 일치	미선택	정상
10	16	예상과 일치	미선택	정상

18. Bitstream 해시 표

실험	Bitstream	SHA-256
1	01_logic_gate.bit	EF7289B...
2	02_full_adder.bit	73FF197...
3	03_adder_4bit.bit	4A1C908...
4	04_sub_4bit.bit	76E9BA8...
5	05_compare_4.bit	06BF8B3...
6	06_mux_4x1.bit	C9097CB...
7	07_demux_1x8.bit	7F5C2A1...
8	08_encoder8x3.bit	F834F91...
9	09_decoder3x8.bit	E429342...
10	10_seg_decoder.bit	7C932B2...

19. 결론

본 실험에서는 마이크로프로세서를 포함해서 조합논리회로 관련 이론 수업에서 배웠던 다양한 회로를 Verilog HDL로 구현하고 시뮬레이션 파형을 얻었다. 나아가 Vivado 2026.1을 이용하여 시뮬레이션, 합성, 구현 및 비트

스트림 생성 과정을 수행하고, 직접 FPGA 보드에 회로를 시연하고, 실제 출력까지 확인하였다, 또한 예상 출력과 비교하여 지금까지 배웠던 조합논리회로를 설계부터 구현까지 하나의 사이클로 해결하는 능력을 기를 수 있었다. 이 과정에서 처음 사용해보았던 Verilog 및 Vivado 프로그램을 익히고, 다양한 실험에 적용할 수 있는 발판을 마련하였다.